

Evaluación globalizadora B

Control de posición de motor DC

Matwiejczuk Leandro Dario

leandromatw@gmail.com

Iván Gonzalo Irumberry

ivangonzalo1996@hotmail.com

Abstract— A menudo, en la industria, surge la necesidad de poder controlar el ángulo u orientación de un eje cuando se lo desee; en estas aplicaciones es necesaria una interfaz comprensible y eficiente, y un dispositivo capaz de controlar un motor de manera precisa. Este proyecto propone una solución a esta necesidad, permitiendo al usuario configurar manualmente el ángulo al que desea se mueva el eje del motor, e incorporando otras ventajas como el uso de indicadores luminosos, display para leer y configurar el equipo de manera cómoda, posibilidad de control desde una computadora, etc. El siguiente proyecto ha sido desarrollado con un microcontrolador STM32F103C8T6 (cortex M3), una Raspberry pi4 (computadora de placa única de tamaño reducido) y utilizando un sistema operativo en tiempo real (FreeRtos v11.1.0), en el entorno de desarrollo IDE STM32CUBEIDE 1.6.0.

Palabras Clave— PWM - PID - Cortex M3 - FreeRTOS - Motor DC - i2C.

I. Índice

I. Índice.....	1
II. Introducción.....	1
III. Fundamentación.....	2
A. Problemática.....	2
B. Solución.....	2
C. Fundamentación técnica del software.....	2
D. Fundamentación técnica del hardware.....	2
IV. Objetivos.....	3

V. Alcance.....	3
VI. Diagrama en bloques.....	3
VII. Diagrama de Software.....	3
VIII. Diagrama de tareas EGA.....	3
IX. Manual de usuario - Comandos.....	3
X. Conclusiones.....	4
a. Porcentaje del alcance logrado.....	4
b. Problemas encontrados.....	4
c. Posibles mejoras.....	4
XI. Referencias.....	4
ANEXO I - Diagrama de tareas EGA.....	5
ANEXO II - Menú.....	6
ANEXO III - Diagrama de bloques.....	6
ANEXO IV - Diagrama de software EGB.....	7
ANEXO V - Consola de usuario EGB.....	7

II. Introducción

El proyecto consiste en realizar un control de posición de un motor DC, siendo este un sistema regulable, además debe de tener la posibilidad de ingresar el ángulo deseado del motor y debe permitir contrastar el ángulo actual del motor con el ángulo deseado. El manejo del motor se hace por medio de pines de entrada y salida (salida en nuestro caso) de propósito general (GPIO) y por medio de una señal PWM de control de velocidad.

III. Fundamentación

A. Problemática

Los motores de corriente continua utilizados por sí mismos no permiten al usuario regular una posición específica a la cuál el eje tiene que ir, solo

permiten el movimiento continuo a una velocidad específica en sentido horario o antihorario. Esto puede probar ser ineficiente en usos donde no se requiera movimiento continuo, sino un solo desplazamiento hasta el ángulo deseado, y en diversos casos es necesario un desplazamiento más o menos abrupto, lo que también supone regulación de velocidad.

B. Solución

Una forma de poder controlar el ángulo actual del eje de un motor es utilizando un encoder magnético, que permite cuantizar el sentido del campo magnético de un imán, utilizando un imán de neodimio adherido químicamente al eje del motor es posible medir la orientación del eje del mismo. Dado que a una tensión de control constante el motor giraría demasiado rápido como para que la propuesta sea viable, se optó por regular su velocidad utilizando una señal modulada por modulación de ancho de pulso (PWM) a la entrada de un puente H, lo que permite variar la velocidad y orientación del motor a voluntad.

Al conectarse a una Raspberry pi se permite el control remoto del dispositivo utilizando una interfaz gráfica guardada en la misma por medio de ssh, donde se pueden controlar, o configurar las variables de funcionamiento.

C. Fundamentación técnica del software

Para la realización se decidió hacer uso del sistema operativo en tiempo real FreeRTOS, implementado en un proyecto escrito en lenguaje C.

El uso de FreeRTOS otorga la posibilidad de organizar cada una de las funciones que debe realizar el equipo en tareas ordenadas por prioridad, permitiendo así su sincronización por medio de recursos específicos del RTOS (sistema operativo en tiempo real). El uso de estos recursos facilita la adquisición de datos, su manejo, y garantiza que las operaciones a realizar tengan un orden específico y se comuniquen entre sí de manera eficiente.

Para el manejo de la señal PWM, se optó por utilizar uno de los timers integrados del dispositivo ya que esto permite que, siempre que se lo desee,

haya una señal PWM activa mientras el procesador realiza otras operaciones.

Para la interfaz de usuario, se optó por utilizar protocolo de comunicación I2C para comunicarse con un display, debido a la considerable reducción de conexiones que permite en comparación con no usar este protocolo. Para la comunicación con el encoder magnético también se utilizó I2C, debido a que una de las ventajas del protocolo es la capacidad de poder conectar varios dispositivos a una misma línea de comunicación y comunicarse con ellos individualmente por medio del uso de su dirección de memoria.

D. Fundamentación técnica del hardware

Para poder controlar el motor de DC, se optó por utilizar un módulo de puente H L298N, cuya entrada recibe señales del microcontrolador.

El microcontrolador STM32F103C8T6 se utilizó como componente principal del proyecto, haciendo uso de la totalidad de la frecuencia disponible como señal de clock general del dispositivo (72Mhz), garantizando un rápido tiempo de ejecución de sentencias y permitiendo utilizar una buena resolución a la hora de configurar el contador (TIM) que creará la señal PWM.

Para poder medir la posición relativa del eje del motor se utilizó módulo encoder magnético basado en el integrado AS5600L, que comunica los valores deseados por medio de protocolo I2C.

La información trabajada se muestra en un display LCD 1604 que posee 4 renglones de 16 caracteres cada uno, así también como permitir visualizar las opciones del menú y sus configuraciones.

El circuito principal del proyecto (EGA) debe de permitir conexión a los pines de la placa bluepill que utilicen el puerto de comunicación serie UART0, a donde se conectó la Raspberry Pi 4.

IV.Objetivos

- **Demostrar uso y conocimiento del hardware utilizado**
 - Realizar coneccionado físico mediante un pcb de bluepill, módulo

de puente H, encoder, display LCD, RTC, fuente regulada de tensión, etc.

- **Configurar periféricos del dispositivo**
 - Configurar, crear o utilizar librerías que permitan hacer uso de I2C, timers (TIM1 y TIM2), GPIO, freertos, etc.
- **Variar la posición del eje del motor**
 - Mediante PWM, utilizando el TIM1, canal 2 en modo PWM, conectado al puente H L298N.
- **Medir la posición angular del eje del motor**
 - Haciendo uso del encoder AS5600L, que envía trama por protocolo I2C.
- **Controlar sentido de giro**
 - Mediante GPIO, conectados al puente H L298N.
- **Regular posición del eje mediante PID**
 - Haciendo uso de teoría del PID, computando valores de entrada y salida del sistema de lazo cerrado Motor - Controlador Motor - Encoder y así variando el ciclo de trabajo de la señal PWM.
- **Led de señalización**
 - Para indicar cuándo el eje del motor está dentro del rango deseado.
- **Mostrar parámetros de control**
 - Mediante display LCD 1604.
- **Proveer una interfaz de usuario**
 - Mediante uso de pulsadores, e interrupciones por GPIO.
 - Mediante aplicación de consola accesible por ssh.

V. Alcance

Se pretende poder medir con una precisión de 0.1° , y permitir al usuario setear el punto a mover el motor, así como el modo de operación:

- **Modo de movimiento brusco:** La velocidad inicial es representada por una función escalón.

- **Modo de movimiento suave:** La velocidad inicial es representada por una función rampa ascendente.

VI. Diagrama en bloques

Los módulos que conforman el equipo se esquematizan a continuación, en donde pueden verse las conexiones entre ellos así como los pines correspondientes a cada conexión.

VER ANEXO III

VII. Diagrama de Software

El sistema se compone de un módulo de kernel dentro de la Raspberry donde se inicializa el driver de UART y 1 char device lo que permite enviar o leer datos por UART por medio de la lectura o escritura del char device.

En user space, se controla mediante ssh una aplicación escrita en Python3 que permite al usuario mediante la interacción con una consola especializada enviar o recibir datos (escribir o leer) de la EGA.

VER ANEXO IV

VIII. Diagrama de tareas EGA

En Anexo I se encuentra el mapa del código que resume la función de cada tarea y su relación con las demás tareas del proyecto.

IX. Manual de usuario - Comandos

A continuación se listan los comandos y sus funciones:

1. Enviar configuración

1. Setpoint :

Envía el Setpoint deseado

2. Error de Setpoint:

Envía el error de Setpoint deseado

3. Elegir modo:

Permite elegir el modo (Rápido o Lento)

4. Regresar al menú anterior:

Regresa al menú principal

2. Recibir información

1. *Setpoint:*
Recibe el Setpoint actual
2. *Error de Setpoint:*
Recibe el error de Setpoint actual
3. *Modo de funcionamiento:*
Recibe el modo actual
4. *Log:*
Recibe el log
5. *Regresar al menú anterior*
Regresa al menú principal
3. *Iniciar funcionamiento*
Inicia el sistema PID
4. *Terminar funcionamiento*
Pausa el sistema PID
5. *Terminar ejecución*
Cierra el programa

X. Conclusiones

a. Porcentaje del alcance logrado

Se logró medir con una precisión aproximada de 0.1° , pero se optó por utilizar una resolución mínima de funcionamiento (configurable) de 1° debido a que valores menores implicarían una modificación a la lógica de funcionamiento y no se habría podido lograr en tiempo y forma.

Se cumplió con la propuesta de permitir al usuario seleccionar un modo de funcionamiento (lento o rápido) por lo que se considerará satisfactoria la realización del proyecto.

b. Problemas encontrados

Durante la realización del proyecto surgieron numerosos problemas debido a tanto mal planeamiento, como falta de conocimiento y/o mala ejecución.

Primeramente, se realizó un error en la impresión del pcb (un integrado se espejó por error) lo que significó un retraso considerable en los tiempos de testeo.

El manejo preciso del motor tuvo muchos inconvenientes, principalmente el lograr mover el eje del mismo al punto deseado sin consumir

muchos recursos del sistema o tardar demasiado tiempo; esto sin contar las numerosas veces que se tuvo que repensar el código o modificar librerías existentes para mantener un funcionamiento en tiempo real sin pausas constantes.

La realización de un menú, aunque era considerada como sencilla por los integrantes del grupo, probó ser un desafío de implementar dadas las necesidades de utilizar numerosas variables de configuración o de aviso en el marco de un sistema operativo en tiempo real, pero se logró finalmente realizarlo.

Finalmente, cabe destacar la complejidad de configurar un controlador PID y de luego calibrarlo, lo que supuso constantes intentos hasta obtener el funcionamiento deseado.

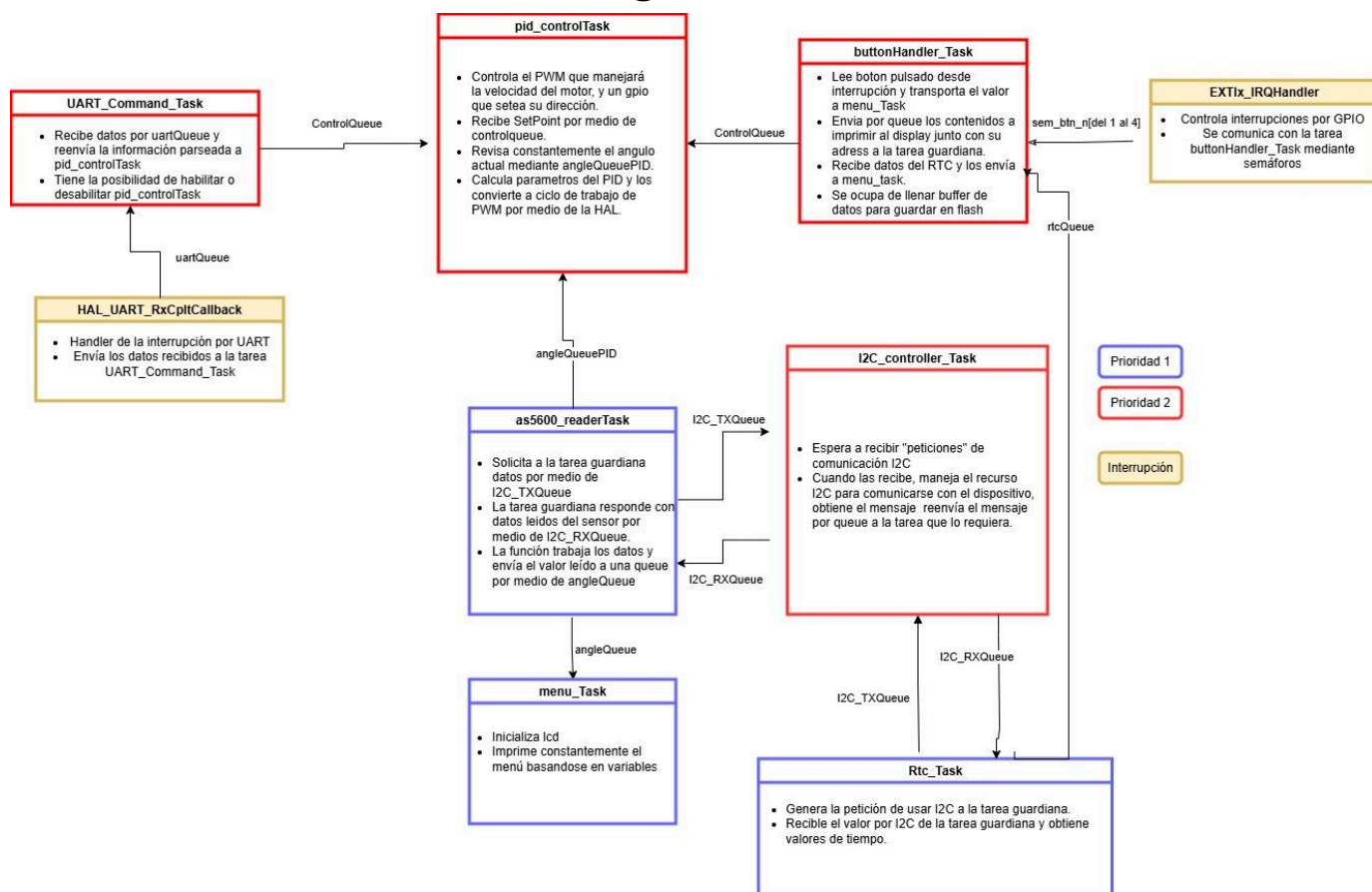
c. Posibles mejoras

Posibles mejoras para este proyecto incluyen principalmente la optimización del controlador PID y reducción del mínimo ángulo configurable. Además se podrían incorporar todos los componentes de los módulos utilizados dentro de una misma placa para minimizar el tamaño del proyecto.

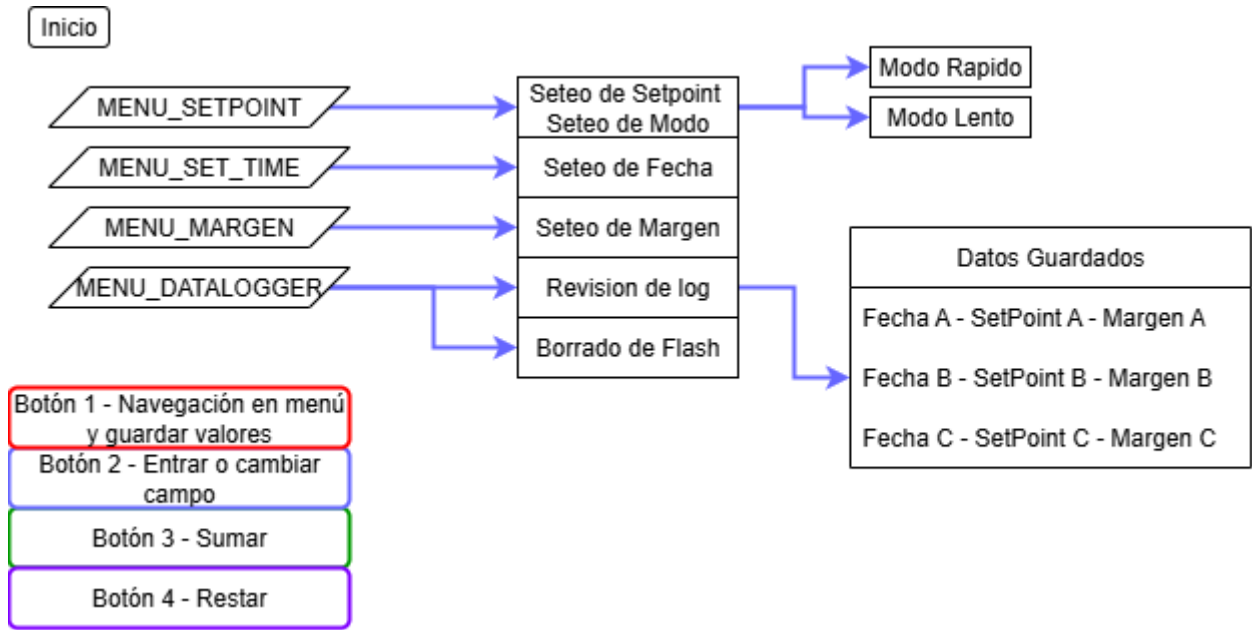
XI. Referencias

- [1] FreeRTOS
<https://www.freertos.org/>
- [2] STM32F103C8T6
<https://www.st.com/en/microcontrollers-microprocessors/stm32f103c8.html>
- [3] Hoja de datos IC encoder AS5600L
<https://ams-osram.com/products/sensor-solutions/position-sensors/ams-as5600l-magnetic-rotary-position-sensor>
- [4] Hoja de datos display WH1604A
<https://www.winstar.com.tw/es/products/lcd-display/character-lcd-display-module/lcd-16x4.html>
- [5] Hoja de datos IC RTC3231
<https://www.digikey.es/es/products/detail/analog-devices-inc-maxim-integrated/DS3231M-TTL/2402421>

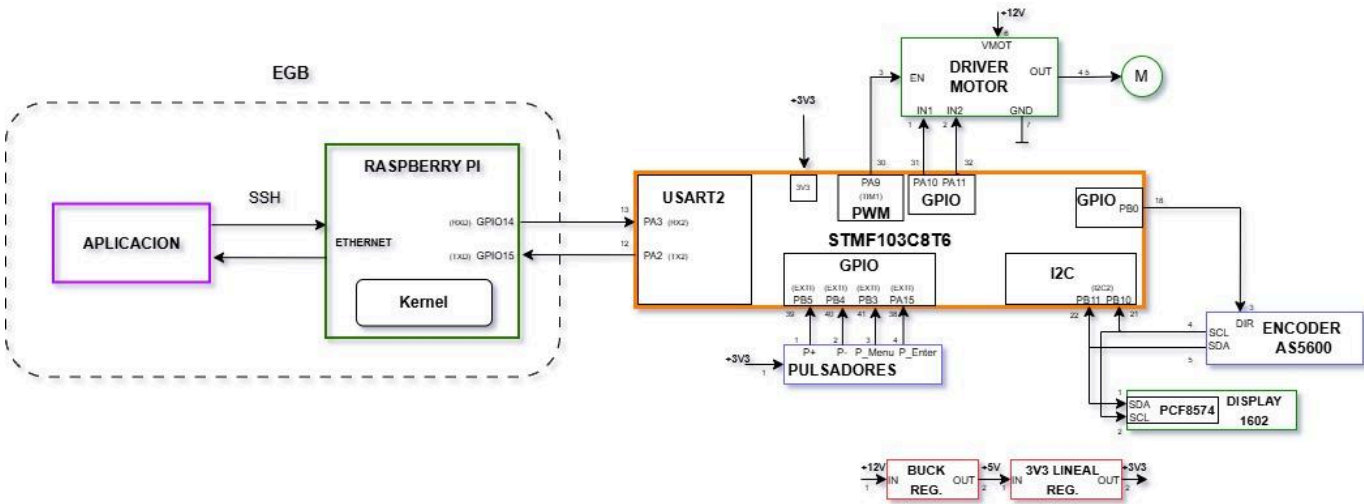
ANEXO I - Diagrama de tareas EGA



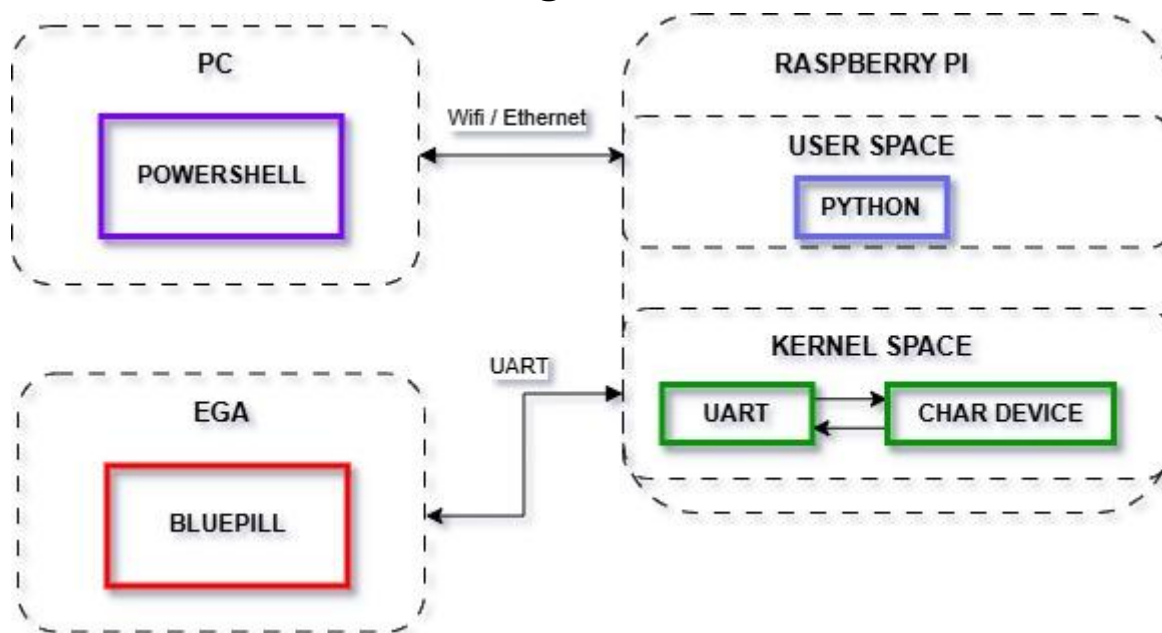
ANEXO II - Menú



ANEXO III - Diagrama de bloques



ANEXO IV - Diagrama de software EGB



ANEXO V - Consola de usuario EGB

```
*****
*****{ User Space UART EGB }*****
*****

Ingrese acción:

1--> Enviar configuración
2--> Recibir información
3--> Iniciar funcionamiento
4--> Terminar funcionamiento
5--> Terminar ejecución
```